



PALADIN
BLOCKCHAIN SECURITY

Smart Contract Security Assessment

Final Report

For HyperCycle (ANFE)

19 February 2025



paladinsec.co



info@paladinsec.co

Table of Contents

Table of Contents	2
Disclaimer	3
1 Overview	4
1.1 Summary	4
1.2 Contracts Assessed	4
1.3 Findings Summary	5
1.3.1 AdvancedNodeFactoryEnclosure	6
2 Findings	7
2.1 AdvancedNodeFactoryEnclosure	7
2.1.1 Privileged Functions	7
2.1.2 Issues & Recommendations	8



Disclaimer

Paladin Blockchain Security ("Paladin") has conducted an independent audit to verify the integrity of and highlight any vulnerabilities or errors, intentional or unintentional, that may be present in the codes that were provided for the scope of this audit. This audit report does not constitute agreement, acceptance or advocacy for the Project that was audited, and users relying on this audit report should not consider this as having any merit for financial advice in any shape, form or nature. The contracts audited do not account for any economic developments that may be pursued by the Project in question, and that the veracity of the findings thus presented in this report relate solely to the proficiency, competence, aptitude and discretion of our independent auditors, who make no guarantees nor assurance that the contracts are completely free of exploits, bugs, vulnerabilities or deprecation of technologies. Further, this audit report shall not be disclosed nor transmitted to any persons or parties on any objective, goal or justification without due written assent, acquiescence or approval by Paladin.

All information provided in this report does not constitute financial or investment advice, nor should it be used to signal that any persons reading this report should invest their funds without sufficient individual due diligence regardless of the findings presented in this report. Information is provided 'as is', and Paladin is under no covenant to the completeness, accuracy or solidity of the contracts audited. In no event will Paladin or its partners, employees, agents or parties related to the provision of this audit report be liable to any parties for, or lack thereof, decisions and/or actions with regards to the information provided in this audit report.

Cryptocurrencies and any technologies by extension directly or indirectly related to cryptocurrencies are highly volatile and speculative by nature. All reasonable due diligence and safeguards may yet be insufficient, and users should exercise considerable caution when participating in any shape or form in this nascent industry.

The audit report has made all reasonable attempts to provide clear and articulate recommendations to the Project team with respect to the rectification, amendment and/or revision of any highlighted issues, vulnerabilities or exploits within the contracts provided. It is the sole responsibility of the Project team to sufficiently test and perform checks, ensuring that the contracts are functioning as intended, specifically that the functions therein contained within said contracts have the desired intended effects, functionalities and outcomes of the Project team.

Paladin retains the right to re-use any and all knowledge and expertise gained during the audit process, including, but not limited to, vulnerabilities, bugs, or new attack vectors. Paladin is therefore allowed and expected to use this knowledge in subsequent audits and to inform any third party, who may or may not be our past or current clients, whose projects have similar vulnerabilities. Paladin is furthermore allowed to claim bug bounties from third-parties while doing so.

1 Overview

This report has been prepared for HyperCycle on the Ethereum network. Paladin provides a user-centred examination of the smart contracts to look for vulnerabilities, logic errors or other issues from both an internal and external perspective.

1.1 Summary

Project Name	HyperCycle
URL	https://www.hypercycle.ai/
Platform	Ethereum
Language	Solidity
Preliminary Contracts	https://github.com/hypercycle-development/hypercycle-contracts/commit/2c9b090995d28c200a5e6792a07c9c58e1fdd840
Resolution 1	https://github.com/hypercycle-development/hypercycle-contracts/commit/dbeda32745c65e31f745552a23ef3798f4199d00

1.2 Contracts Assessed

Name	Contract	Live Code Match
AdvancedNodeFacto		
ryEnclosure		

1.3 Findings Summary

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)
● Governance	0	-	-	-
● High	3	3	-	-
● Medium	1	1	-	-
● Low	6	6	-	-
● Informational	4	4	-	-
Total	14	14	-	-

Classification of Issues

Severity	Description
● Governance	Issues under this category are where the governance or owners of the protocol have certain privileges that users need to be aware of, some of which can result in the loss of user funds if the governance's private keys are lost or if they turn malicious, for example.
● High	Exploits, vulnerabilities or errors that will certainly or probabilistically lead towards loss of funds, control, or impairment of the contract and its functions. Issues under this classification are recommended to be fixed with utmost urgency.
● Medium	Bugs or issues with that may be subject to exploit, though their impact is somewhat limited. Issues under this classification are recommended to be fixed as soon as possible.
● Low	Effects are minimal in isolation and do not pose a significant danger to the project or its users. Issues under this classification are recommended to be fixed nonetheless.
● Informational	Consistency, syntax or style best practices. Generally pose a negligible level of risk, if any.

1.3.1 AdvancedNodeFactoryEnclosure

ID	Severity	Summary	Status
01	HIGH	Adding licence tokens to an already minted ANFE distorts the order of registered assets for that ANFE	✓ RESOLVED
02	HIGH	ANFE deposit restriction is not enforced	✓ RESOLVED
03	HIGH	Tokens can be added to non-minted ANFEs and distort asset order	✓ RESOLVED
04	MEDIUM	ANFE.numAssets not reduced when redeeming licence token	✓ RESOLVED
05	LOW	transfer is used instead of safeTransfer for ERC20 tokens	✓ RESOLVED
06	LOW	Fee-on-transfer/rebasing tokens are not supported	✓ RESOLVED
07	LOW	No limit on the burn message size	✓ RESOLVED
08	LOW	CEI pattern not adhered to	✓ RESOLVED
09	LOW	Not all fields are nullified upon ANFE being burned	✓ RESOLVED
10	LOW	The openDeposit and delegateSigner are not reset on transfer	✓ RESOLVED
11	INFO	Use Ownable2Step as a safer alternative to Ownable	✓ RESOLVED
12	INFO	Typographical issues	✓ RESOLVED
13	INFO	Insufficient validation	✓ RESOLVED
14	INFO	Gas optimizations	✓ RESOLVED

2 Findings

2.1 AdvancedNodeFactoryEnclosure

AdvancedNodeFactoryEnclosure allows users to group a HyperCycle license NFT along with multiple ERC20s/ERC721s/ERC1155s into a single, transferable NFT bundle called an Advanced Node Factory Enclosure, or ANFE.

The owner of a licence token provides it to the contract and receives a minted ANFE ERC721 token. Different types of assets can be deposited for minted ANFEs, which can later be withdrawn by the ANFE owner.

Each ANFE owner has the option to trade it back in the protocol for the original licence token that was provided or they can directly burn the ANFE and lock the licence token.

2.1.1 Privileged Functions

- `createAdvancedNodeFactoryEnclosure[OWNER]`
- `renounceOwnership [OWNER]`
- `transferOwnership [OWNER]`

2.1.2 Issues & Recommendations

Issue #01	Adding licence tokens to an already minted ANFE distorts the order of registered assets for that ANFE
------------------	--

Severity

 **HIGH SEVERITY**

Description

`createAdvancedNodeFactoryEnclosure()` registers the provided licence token always with the assumption that it is the first ERC721 token (at index zero 0) inside the `_assetKeys` storage mapping when `_addAssetData()` is called. Depending on whether the ANFE for the provided licence token has already been minted, the ANFE is either minted or transferred out to the owner of the licence token.

Once the owner receives the ANFE token, they can accumulate different asset types that have been deposited to it and later withdraw them. The ANFE owner can also call `tradeIn()` which swaps the ANFE token for the initially deposited licence token and sends it back to the user. The issue arises when the traded licence token is subsequently used within `createAdvancedNodeFactoryEnclosure()` to retrieve the ANFE token again.

PoC:

1. Bob owns ANFE_1 which has [LICENCE_NFT, NFT_1 & NFT_2] in its `_assetKeys[LICENCE_NFT][ERC721]` array (licence token is put at index 0).
2. Bob calls `tradeIn()` with `overrideTrade=true`, which allows tokens not to be withdrawn, and LICENCE_NFT is transferred to the owner in exchange for ANFE_1. As a result, `_assetKeys[LICENCE_NFT][ERC721]` is updated as [NFT_2, NFT_1] due to the swap/pop done in the call.
3. `createAdvancedNodeFactoryEnclosure()` is called again with LICENCE_NFT for ANFE_1 and as result the licence is NOT added as first element, since the `_assetKeys ERC721` array is not empty. The result is [NFT_2, NFT_1, LICENCE_NFT], where the licence NFT is put at the last position.
4. Now the 0 index check that prevents direct withdrawals of the licence token within `withdrawAsset()` does not work and the owner can directly withdraw LICENCE_NFT at index 2.

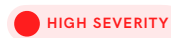
Recommendation When creating ANFEs that have already been minted, if the ERC721 `_assetKeys` array is not empty, make sure to insert the licence NFT at index 0 and move the element from index 0 at the end. This way, the licence NFT would always be at the start, and the protocol assumptions would be respected.

Resolution



Issue #02 **ANFE deposit restriction is not enforced**

Severity



Description

The ANFE owner can configure the `openDeposit` variable which controls if non-owners can add assets into the ANFE. However, the `addAsset()` function never checks the `openDeposit` variable, meaning that everyone will always be able to add assets to an ANFE.

Recommendation At the start of `addAsset()`, check the status of the `openDeposit` variable and if it is disabled, only allow the owner to call it.

Resolution



Issue #03**Tokens can be added to non-minted ANFEs and distort asset order****Severity** HIGH SEVERITY**Description**

`addAsset()` never checks if the ANFE for the provided licence id has been minted.

This allows ERC721 tokens to be added to the ERC721 `_assetKeys` mapping of the ANFE before it is minted. Thus, when it gets minted, the licence NFT would not be placed at index 0 but pushed at the end of the ERC721 array.

The impact of this is similar to the one explained in a previous issue - bypassing the `index(0)` check when withdrawing tokens.

Additionally, the function never checks if the ANFE for the provided licence id has been burned. Adding tokens for such ANFEs just locks them in the contract and should not be allowed.

Recommendation

Consider adding checks that ensure the ANFE token for which assets are being added is in valid state:

```
if (!_ANFEData[licenseId].minted) revert ANFENotMinted
```

And also check that it is not burned:

```
if (_ANFEData[licenseId].burned) revert ANFEBurned
```

Resolution RESOLVED

Issue #04	ANFE.numAssets not reduced when redeeming licence token
Severity	 MEDIUM SEVERITY
Description	Within <code>tradeIn()</code> , when the licence token at index 0 is removed, the <code>_ANFEData[licenseId].numAssets</code> field of the ANFE struct is not reduced although it is incremented when ANFE is created.
Recommendation	Consider decrementing <code>numAssets</code> by 1 in <code>tradeIn()</code> .
Resolution	 RESOLVED

Issue #05	transfer is used instead of safeTransfer for ERC20 tokens
Severity	 LOW SEVERITY
Description	Within <code>addAsset()</code> and <code>withdrawAsset()</code>, ERC20 transfers are executed by calling <code>transfer()</code>. It is always recommended to use OpenZeppelin's <code>safeTransfer</code> library as it handles interactions with non-standard tokens (such as USDT) which would otherwise revert through <code>transfer()</code>.
Recommendation	Consider using <code>safeTransfer()</code> instead of <code>transfer()</code> when sending ERC20 tokens.
Resolution	 RESOLVED



Issue #06 Fee-on-transfer/rebasing tokens are not supported	
Severity	 LOW SEVERITY
Description	The contract logic is not capable of handling fee-on-transfer or rebasing tokens as they change their balances. Since any token can be added to an ANFE, this will DOS withdraws and cause unintended behavior.
Recommendation	If no whitelisting is implemented, consider informing users that such tokens would not work.
Resolution	 RESOLVED

Issue #07 No limit on the burn message size	
Severity	 LOW SEVERITY
Description	When calling <code>burnANFE()</code>, the owner can provide a message string. However, no limit is enforced on its length, which will impact the <code>ANFEData</code> struct that is being read and can potentially cause DoS.
Recommendation	Consider implementing a reasonable length limit to the burn message.
Resolution	 RESOLVED

Issue #08	CEI pattern not adhered to
Severity	 LOW SEVERITY
Description	When calling <code>createAdvancedNodeFactoryEnclosure()</code>, external calls are made before state variables are updated. It is always considered a best practice to execute external calls only once all storage has been updated In this case, make sure to first update the <code>openDeposit</code> and <code>minted</code> fields of <code>_ANFEData</code> and only execute the <code>safeTransfer/safeMint</code> calls after that.
Recommendation	Move the <code>openDeposit</code> update before the <code>transfer/mint</code> calls and also update the <code>minted</code> field before <code>safeMint()</code> , not after it.
Resolution	 RESOLVED

Issue #09	Not all fields are nullified upon ANFE being burned
Severity	 LOW SEVERITY
Description	When burning ANFE tokens, the following fields of <code>_ANFEData[licenseId]</code> should also be nullified: - <code>openDeposit</code> - <code>delegatedSigner</code> Since the ANFE does not exist any more, it makes sense to reset them.
Recommendation	Consider resetting the fields.
Resolution	 RESOLVED

Issue #10 The openDeposit and delegateSigner are not reset on transfer	
Severity	 LOW SEVERITY
Description	The ANFE is an ERC721 token, and the best practice when transferring this type of token is to reset any previous approvals/grants to prevent unintended behavior by previously granted parties. Using the same logic, the signer and open deposits should also be reset upon transfer. This ensures that the new owner can define their own signer and open deposit strategy, avoiding any potential misuse or conflicts from the previous owner's settings.
Recommendation	Consider resetting these fields by overwriting the _update function. We can draw inspiration from how approvals are reset upon transfer.
Resolution	 RESOLVED

Issue #11 Use Ownable2Step as a safer alternative to Ownable	
Severity	 INFORMATIONAL
Description	The contract inherits Ownable for access control. A safer alternative is to use Ownable2Step — rather than directly transferring to the new owner, the transfer only completes when the new owner accepts explicitly the ownership.
Recommendation	Consider using Ownable2Step instead of Ownable. This makes ownership transfers much safer and prevents accidental transfer to invalid addresses
Resolution	 RESOLVED

Description

The following import is not used and can be removed:

```
import '@openzeppelin/contracts/utils/introspection/ERC165.sol';
```

—

In the dev comment at the top that explains the logic of the AdvancedNodeFactoryEnclosureV1 contract, there is a typographical error in the following sentence at line 65:

"... the option to set a delegatedSigner, that can used to start node tilling..." is missing *be*: "...can *be* used to start node tilling..."

—

In the description of the assetType parameter of `_addAssetData()` and `_removeAssetData()`, there is a typographical error: "the type of asseting.." should be "the type of asset.."

—

At the error description on L39, "deposting" should be "depositing".

—

At the error description on L48, "Trown" should be "Thrown".

—

Within `burnANFE()`, consider reusing the `isOwner` modifier instead of the following check:

```
if (ownerOf(licenseId) != msg.sender) {  
    revert MustBeANFEOwner();  
}
```

—

Missing NatSpecs:

- `createAdvancedNodeFactoryEnclosure` is missing return parameter NatSpec
- `supportsInterface` missing NatSpec

Incomplete or missing events:

- ANFECreated event does not contain openDeposit
- ANFELicenseTradeIn does not contain overrideTrade
- ANFEBurned does not contain overrideBurn

Recommendation Consider implementing the above recommendations.

Resolution  RESOLVED

Issue #13 **Insufficient validation**

Severity  INFORMATIONAL

Description Inside the constructor, check that the provided hypercycleLicenseAddress address is not address(0).

—

Inside createAdvancedNodeFactoryEnclosure, check that licenseId is not 0. The initial token id is START_ROOT_TOKEN, so 0 should not be allowed. This applies for addAsset() as well.

—

Inside addAsset(), validate that contractAddress is not address(0).

Recommendation Consider implementing the above recommendations.

Resolution  RESOLVED

Severity

 INFORMATIONAL

Description

Inside `createAdvancedNodeFactoryEnclosure()`, move the licence token transfer before `_addAssetData()` to skip unnecessary operations in case of revert.

—

The `ANFEData` struct can be rewritten for a more packed version that it will save gas:

```
solidity
    struct ANFEData {
        address delegatedSigner;
        bool burned;
        bool minted;
        bool openDeposit;
        uint256 numAssets;
        string message;
    }
```

Moving second to last will make the above fields be packed into one slot.

—

The `IHypercycleLicense` should be constant and public.

—

Consider using named return variables in functions to reduce gas usage:

e.g. `function() returns(uint256)` can be `function() returns(uint256 namedVar)`

Recommendation

Consider implementing the above recommendations.

Resolution

 RESOLVED



PALADIN
BLOCKCHAIN SECURITY